



### Alunos da equipe:

- Amanda Neumann
- Gustavo Gomes Marcondes
- Helen Sara Teixeira
- Igor Nathan Lobato
- Jean Roberto Reinert
- Matheus Henrique Miranda

Seed utilizado: 202650

(Ano atual com 4 dígitos + 2 algarismos do dígito verificador do CPF de um dos integrantes)

### Especificações:

O trabalho pode ser feito por uma equipe de 1 a 6 integrantes.

Para cada problema, preencher as colunas dos quadros com o que pede.

Além disso, fazer as solicitações pedidas antes dos quadros.

## CLASSIFICAÇÃO

Para o experimento de Classificação:

- Ordenar pela Acurácia (descendente), ou seja, a técnica de melhor acurácia ficará em primeiro na tabela.
- Após o quadro colocar:
  - Um resultado com 3 linhas com a predição de novos casos para a técnica/parâmetro de maior Acurácia (criar um arquivo com novos casos à sua escolha)
  - A lista de comandos emitidos no RStudio para conseguir os resultados obtidos

### Veículo

| Técnica | Parâmetro | Acurácia | Matriz de Confusão |
|---------|-----------|----------|--------------------|
|---------|-----------|----------|--------------------|

|                |                   |        |                                                                                                             |
|----------------|-------------------|--------|-------------------------------------------------------------------------------------------------------------|
| SVM – CV       | C=100 Sigma=0.015 | 0.8024 | <pre> Reference Prediction bus opel saab van bus 42 0 0 1 opel 0 28 16 1 saab 0 13 27 0 van 1 1 0 37 </pre> |
| RNA – CV       | size=11 decay=0.4 | 0.7784 | <pre> Reference Prediction bus opel saab van bus 41 0 1 0 opel 0 29 19 1 saab 1 12 23 1 van 1 1 0 37 </pre> |
| SVM – Hold-out | C=1 Sigma=0.0775  | 0.7605 | <pre> Reference Prediction bus opel saab van bus 42 1 1 0 opel 0 19 13 0 saab 0 20 27 0 van 1 2 2 39 </pre> |
| RF – CV        | mtry=5            | 0.7545 | <pre> Reference Prediction bus opel saab van bus 43 1 2 0 opel 0 22 16 0 saab 0 17 22 0 van 0 2 3 39 </pre> |
| RF – Hold-out  | mtry=2            | 0.7305 | <pre> Reference Prediction bus opel saab van bus 41 1 1 0 opel 0 22 19 0 saab 1 16 20 0 van 1 3 3 39 </pre> |
| RNA – Hold-out | size=5 decay=0.1  | 0.6946 | <pre> Reference Prediction bus opel saab van bus 38 0 1 3 opel 0 7 5 1 saab 4 35 36 0 van 1 0 1 35 </pre>   |
| KNN            | k=1               | 0.6527 | <pre> Reference Prediction bus opel saab van bus 41 2 3 3 opel 0 17 20 3 saab 2 20 19 1 van 0 3 1 32 </pre> |

Tabela

| Comp                                | Circ | DCirc | RadRa | PrAssila | MaxRa | ScatRa | Elong | PrAssilct | MaxRect | ScVarMaxis | ScVarMaxis | RaGrp | SkewMaxis | Skewmaxis | KurtMaxis | KurtMaxis | HollRa | Predicao |
|-------------------------------------|------|-------|-------|----------|-------|--------|-------|-----------|---------|------------|------------|-------|-----------|-----------|-----------|-----------|--------|----------|
| 101                                 | 56   | 100   | 215   | 69       | 10    | 208    | 32    | 24        | 169     | 227        | 651        | 223   | 74        | 6         | 5         | 186       | 193    | opel     |
| 81                                  | 45   | 68    | 169   | 73       | 6     | 151    | 44    | 19        | 146     | 173        | 336        | 186   | 75        | 7         | 0         | 183       | 189    | bus      |
| 85                                  | 39   | 68    | 119   | 52       | 5     | 128    | 53    | 18        | 135     | 146        | 241        | 142   | 75        | 8         | 8         | 182       | 187    | van      |
| Kurtmaxis KurtMaxis HollRa Predicao |      |       |       |          |       |        |       |           |         |            |            |       |           |           |           |           |        |          |
| 1                                   |      |       |       | 5        |       | 186    |       | 193       |         | opel       |            |       |           |           |           |           |        |          |
| 2                                   |      |       |       | 0        |       | 183    |       | 189       |         | bus        |            |       |           |           |           |           |        |          |
| 3                                   |      |       |       | 8        |       | 182    |       | 187       |         | van        |            |       |           |           |           |           |        |          |

Código R

```
# install.packages("caret")
# install.packages("e1071")
# install.packages("mlbench")
# install.packages("mice")
# install.packages("randomForest")
# install.packages("nnet")
# install.packages("kernlab")

library(caret)
library(e1071)
library(mlbench)
library(mice)
library(randomForest)
library(nnet)
library(kernlab)

### 1. Obter e preparar os dados
dados <- read.csv("~/UFPR_Conteudo/IAA002-Linguagem de Programacao
Aplicada/trabalho/iaa002-trabalho-python/TRABALHO_DE_IAA008/06 - Veículos/6 -
Veiculos - Dados.csv")

dados$a <- NULL
colnames(dados)[colnames(dados) == "tipo"] <- "y"
dados$y <- as.factor(dados$y)

### 2. Criar bases de Treino e Teste
set.seed(202650)
indices <- createDataPartition(dados$y, p=0.80, list=FALSE)
treino <- dados[indices,]
teste <- dados[-indices,]

### =====
### KNN
### =====
tuneGrid <- expand.grid(k = c(1,3,5,7,9))
set.seed(202650)
knn <- train(y ~ ., data = treino, method = "knn", tuneGrid=tuneGrid)
knn
predict.knn <- predict(knn, teste)
confusionMatrix(predict.knn, as.factor(teste$y))

### =====
### REDES NEURAIS (RNA)
### =====
```

```
### RNA - Hold-out
set.seed(202650)
rna <- train(y~, data=treino, method="nnet", trace=FALSE)
rna
predict.rna <- predict(rna, teste)
confusionMatrix(predict.rna, as.factor(teste$y))

### RNA - Cross-Validation (CV)
ctrl <- trainControl(method = "cv", number = 10)
grid <- expand.grid(size = seq(from = 1, to = 35, by = 10), decay = seq(from = 0.1, to = 0.6, by = 0.3))
set.seed(202650)
rna <- train(form = y~, data = treino, method = "nnet", tuneGrid = grid, trControl = ctrl,
maxit = 2000, trace=FALSE)
rna
predict.rna <- predict(rna, teste)
confusionMatrix(predict.rna, as.factor(teste$y))a

### =====
### SVM
### =====
### SVM - Hold-out
set.seed(202650)
svm <- train(y~, data=treino, method="svmRadial")
svm
predict.svm <- predict(svm, teste)
confusionMatrix(predict.svm, as.factor(teste$y))

### SVM - Cross-Validation (CV)
ctrl <- trainControl(method = "cv", number = 10)
tuneGrid = expand.grid(C=c(1, 2, 10, 50, 100), sigma=c(.01, .015, 0.2))
set.seed(202650)
svm <- train(y~, data=treino, method="svmRadial", trControl=ctrl, tuneGrid=tuneGrid)
svm
predict.svm <- predict(svm, teste)
confusionMatrix(predict.svm, as.factor(teste$y))

### =====
### RANDOM FOREST (RF)
### =====
### RF - Hold-out
set.seed(202650)
rf <- train(y~, data=treino, method="rf")
rf
```

```
predict.rf <- predict(rf, teste)
confusionMatrix(predict.rf, as.factor(teste$y))

### RF - Cross-Validation (CV)
ctrl <- trainControl(method = "cv", number = 10)
tuneGrid = expand.grid(mtry=c(2, 5, 7, 9))

set.seed(202650)
rf <- train(y~, data=treino, method="rf", trControl=ctrl, tuneGrid=tuneGrid)
rf
predict.rf <- predict(rf, teste)
confusionMatrix(predict.rf, as.factor(teste$y))

### =====
### PREDIÇÕES DE NOVOS CASOS (Maior Acurácia: SVM – CV)
### =====
# cria um arquivo com novos casos
casos_ficticios <- dados[c(20, 40, 60), ]
casos_ficticios$y <- NULL
write.csv(casos_ficticios, "~/UFPR_Conteudo/IAA002-Linguagem de Programacao
Aplicada/trabalho/iaa002-trabalho-python/TRABALHO_DE_IAA008/06 - Veículos/6 -
Veiculos - Novos Casos.csv", row.names = FALSE)

dados_novos_casos <- read.csv("~/UFPR_Conteudo/IAA002-Linguagem de Programacao
Aplicada/trabalho/iaa002-trabalho-python/TRABALHO_DE_IAA008/06 - Veículos/6 -
Veiculos - Novos Casos.csv")

predict_novos <- predict(svm, dados_novos_casos)
resultado <- cbind(dados_novos_casos, Predicao = predict_novos)

View(resultado)
print(resultado[, c((ncol(resultado)-3):ncol(resultado))])
print(resultado[, c("Comp", "Circ", "Predicao")])
```

## Diabetes

| Técnica        | Parâmetro              | Acurácia | Matriz de Confusão                           |
|----------------|------------------------|----------|----------------------------------------------|
| SVM – Hold-out | C=0.5 Sigma= 0.1181552 | 0.817    | Prediction neg pos<br>neg 88 16<br>pos 12 37 |
| SVM – CV       | C=1 Sigma=0.1181552    | 0.8039   | Prediction neg pos<br>neg 85 15              |

|                |                  |        |                                              |
|----------------|------------------|--------|----------------------------------------------|
|                |                  |        | pos 15 38                                    |
| RNA – Hold-out | size=3 decay=0.1 | 0.7778 | Prediction neg pos<br>neg 83 17<br>pos 17 36 |
| RF – CV        | mtry=2           | 0.7778 | Prediction neg pos<br>neg 82 16<br>pos 18 37 |
| RF – Hold-out  | mtry=2           | 0.7712 | Prediction neg pos<br>neg 82 17<br>pos 18 36 |
| KNN            | k=9              | 0.7143 | Prediction neg pos<br>neg 84 34<br>pos 10 26 |
| RNA – CV       | size=1 decay=0.7 | 0.6013 | Prediction neg pos<br>neg 88 49<br>pos 12 4  |

**Conclusão:** Como conclusão de nossas análises para a base de diabetes o modelo que obteve os melhores resultados (tendo como base a acurácia) foi a SVM – Hold-out, de  $C=0.5$   $\Sigma=0.1181552$ .

Novos casos:

| preg0nt | glucose | pressure | triceps | insulin | mass | pedigree | age | predict.svm |
|---------|---------|----------|---------|---------|------|----------|-----|-------------|
| 7       | 91      | 74       | 0       | 0       | 18.6 | 0.254    | 31  | neg         |
| 1       | 122     | 90       | 51      | 220     | 18.7 | 0.325    | 31  | neg         |
| 1       | 163     | 72       | 0       | 0       | 39.0 | 1.222    | 33  | pos         |

#### Lista de comandos utilizados em R:

SVM

###Pacotes necessários:

```
install.packages("e1071")
```

```
install.packages("kernlab")
```

```
install.packages("caret")
```

```
library("caret")
```

```
install.packages("mice")
```

```
library(mice)
```

###Leitura dos dados (RStudio online)

```
X10_Diabetes_Dados <- read_csv("10 - Diabetes - Dados.csv")
```

###Criar bases de Treino e Teste

```
set.seed(202650)
indices <- createDataPartition
(X10_Diabetes_Dados$diabetes, p=0.80,list=FALSE)
treino <- X10_Diabetes_Dados[indices,]
teste <- X10_Diabetes_Dados[-indices,]

###Treinar SVM com a base de Treino

set.seed(202650)
svm <- train(diabetes~., data=treino, method="svmRadial")
svm

###Aplicar modelos treinados na base de Teste

predict.svm <- predict(svm, teste)
confusionMatrix(predict.svm, as.factor(teste$diabetes))

###PREDIÇÕES DE NOVOS CASOS

dados_novos_casos <- read.csv("Diabetes - Novos Casos.csv")
dados_novos_casos$num <- NULL View(dados_novos_casos)
predict.svm <- predict(svm, dados_novos_casos)
resultado <- cbind(dados_novos_casos, predict.svm)
resultado$diabetes <- NULL
View(resultado)
```

## REGRESSÃO

Para o experimento de Regressão:

- Ordenar por R2 decrescente, ou seja, a técnica de melhor R2 ficará em primeiro na tabela.
- Após o quadro, colocar:
  - Um resultado com 3 linhas com a predição de novos casos para a técnica/parâmetro de maior R2 (criar um arquivo com novos casos à sua escolha)
  - O Gráfico de Resíduos para a técnica/parâmetro de maior R2
  - A lista de comandos emitidos no RStudio para conseguir os resultados obtidos

## Admissão

| Técnica | Parâmetro | R2 | Syx | Pearson | Rmse | MAE |
|---------|-----------|----|-----|---------|------|-----|
|---------|-----------|----|-----|---------|------|-----|

|                |                           |           |            |           |            |            |
|----------------|---------------------------|-----------|------------|-----------|------------|------------|
| RNA – Hold-out | size=5 decay=0.1          | 0.8128197 | 0.06335154 | 0.9021599 | 0.06313859 | 0.04560445 |
| RF – Hold-out  | mtry=3                    | 0.7913435 | 0.06688722 | 0.8896162 | 0.06666239 | 0.04686918 |
| RF – CV        | mtry=2                    | 0.7855207 | 0.06781408 | 0.8866402 | 0.06758613 | 0.0476019  |
| SVM – CV       | C=1<br>Sigma=0.1981359    | 0.7712667 | 0.07003124 | 0.8819231 | 0.06979584 | 0.05011257 |
| SVM – Hold-out | C=0.25<br>Sigma=0.1981359 | 0.7586477 | 0.07193709 | 0.8801595 | 0.07169529 | 0.05182392 |
| RNA – CV       | size=5 decay=0            | 0.7484108 | 0.07344684 | 0.8676302 | 0.07319996 | 0.05185753 |
| KNN            | K=5                       | 0.7445719 | 0.07400507 | 0.8643849 | 0.07375632 | 0.05423714 |

**Conclusão:** Como conclusão de nossas análises para a base de admissão o modelo que obteve os melhores resultados (tendo como base o parâmetro R2) foi a RNA – Hold-out, de size=5 e decay=0.1.

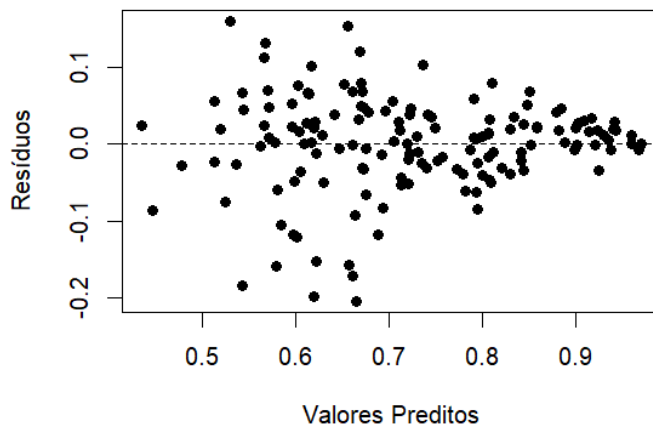
Novos casos:

| Previsão de novos casos - RNA - Hold-out |             |                   |     |     |      |          |               |
|------------------------------------------|-------------|-------------------|-----|-----|------|----------|---------------|
| GRE Score                                | TOEFL Score | University Rating | SOP | LOR | CGPA | Research | ChanceOfAdmit |
| 212                                      | 118         | 4                 | 4,5 | 4,5 | 7,65 | 1        | 0.48014748    |
| 324                                      | 58          | 4                 | 4   | 4,5 | 2,87 | 1        | 0.06974302    |
| 200                                      | 104         | 3                 | 3   | 3,5 | 8    | 1        | 0.40046147    |

Gráfico de Resíduos:



**Gráfico de Resíduos - RNA Hold-out**



**Comandos utilizados em R:**

```
library(caret)
library(Metrics)
library(nnet)

set.seed(202650)

### Leitura dos dados
df <- read.csv("C:/Users/igorn/OneDrive/Documentos/IAA/IAA008/9 - Admissao - Dados(in).csv")
head(df)
str(df)
summary(df)

### Remove num
df$num <- NULL
head(df)

### Separa X e Y
y <- df$ChanceOfAdmit
X <- df[, names(df) != "ChanceOfAdmit"]
### Normaliza os dados
scaler <- preProcess(X, method = "range")
X_scaled <- predict(scaler, X)
head(X_scaled)

### Separa os dados de treino e teste
indice <- createDataPartition(
  y,
  p = 0.7,
```

```
list = FALSE
)
X_train <- X_scaled[indice, ]
X_test <- X_scaled[-indice, ]
y_train <- y[indice]
y_test <- y[-indice]

### RNA - HOLD-OUT
rna <- train(
  x = X_train,
  y = y_train,
  method = "nnet",
  linout = TRUE,
  trace = FALSE
)
rna

### Predição no conjunto de teste
pred_rna <- predict(
  rna,
  X_test
)

### MÉTRICAS
obs <- y_test
pred <- pred_rna
n <- length(obs)
gl <- 1
mae_val <- mae(obs, pred)
rmse_val <- rmse(obs, pred)
r_pearson <- cor(obs, pred)
syx <- sqrt(sum((obs - pred)^2) / (n - gl))
r2 <- function(predito, observado) {
  return(
    1 - (
      sum((predito - observado)^2) /
      sum((observado - mean(observado))^2)
    )
  )
}
r2_val <- r2(pred, obs)

### Consolidação das métricas
resultados_rna <- data.frame(
  R2 = r2_val,
  Syx = syx,
  Pearson = r_pearson,
  RMSE = rmse_val,
```

```
MAE = mae_val
)
print(resultados_rna)

#####
### PREVISÃO DOS NOVOS CASOS
novos_casos <- data.frame(
  `GRE Score` = c(212, 324, 200),
  `TOEFL Score` = c(118, 58, 104),
  `University Rating` = c(4, 4, 3),
  SOP = c(4.5, 4, 3),
  LOR = c(4.5, 4.5, 3.5),
  CGPA = c(7.65, 2.87, 8),
  Research = c(1, 1, 1)
)
print(novos_casos)

### Aplica o scaler utilizado no treinamento
novos_casos_scaled <- predict(
  scaler,
  novos_casos
)
print(novos_casos_scaled)

### Faz as novas previsões
predicoes_novos <- predict(
  rna,
  novos_casos_scaled
)
print(predicoes_novos)

### Adiciona as previsões aos casos
resultado_novos <- novos_casos
resultado_novos$ChanceOfAdmit <- predicoes_novos

### Mostra o resultado final
print(resultado_novos)

### GRÁFICO DE RESÍDUOS
residuos_rna <- y_test - pred_rna

plot(
  pred_rna,
  residuos_rna,
  main = "Gráfico de Resíduos - RNA Hold-out",
  xlab = "Valores Preditos",
  ylab = "Resíduos",
  pch = 19
```

```
)  
  
abline(  
  h = 0,  
  lty = 2  
)
```

## Biomassa

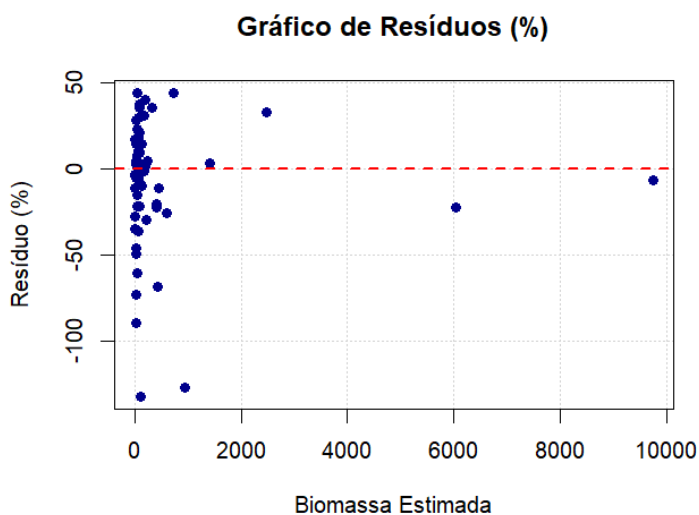
| Técnica        | Parâmetro             | R2        | Syx      | Pearson   | Rmse     | MAE      |
|----------------|-----------------------|-----------|----------|-----------|----------|----------|
| RNA – CV       | size=10<br>decay=0.4  | 0.9736218 | 226.2812 | 0.9867698 | 224.3876 | 96.39921 |
| RF – CV        | mtry=2                | 0.966434  | 255.2561 | 0.9861374 | 253.12   | 93.9897  |
| RF – Hold-out  | mtry=2                | 0.9625793 | 269.5144 | 0.9842475 | 267.2591 | 95.60474 |
| RNA – Hold-out | size=3<br>decay=0.1   | 0.9594064 | 280.7082 | 0.9809546 | 278.3591 | 122.8404 |
| SVM – CV       | C=100<br>Sigma=0.01   | 0.9205996 | 392.5885 | 0.9826738 | 389.3032 | 126.1763 |
| KNN            | K=1                   | 0.8164216 | 596.9487 | 0.9424793 | 591.9532 | 163.776  |
| SVM – Hold-out | C=1<br>Sigma=1.016905 | 0.3654367 | 1109.848 | 0.7149252 | 1100.561 | 296.2975 |

**Conclusão:** A melhor técnica (tendo como base o parâmetro R2) foi a RNA com Cross-Validation, de size=10 e decay=0.4. Nota: Adicionalmente, foi usado um tunegrid para otimizar os hiperparâmetros.

### Novos Casos:

| dap   | h     | Me   | predict.rna |
|-------|-------|------|-------------|
| 9.50  | 11.80 | 0.50 | 36.48596    |
| 15.39 | 13.62 | 0.58 | 71.31044    |
| 19.52 | 24.06 | 0.87 | 633.26455   |

### Gráfico de Resíduos:



#### Lista de Comandos no R:

```
### Instalação dos pacotes
#install.packages("caret")
#install.packages("e1071")
#install.packages("mlbench")
#install.packages("mice")
library(mlbench)
library(caret)
library(mice)

### Leitura dos dados
setwd("~/IAA008 Aprendizado de Maquina/dados/05 - Biomassa")
dados <- read.csv("5 - Biomassa - Dados.csv", header=T)
View(dados)

### Cria arquivo de treino e teste
set.seed(202650)
indices <- createDataPartition(dados$biomassa, p=0.80, list=FALSE)
treino <- dados[indices,]
teste <- dados[-indices,]

### Treino com CV
```

```
set.seed(202650)
control <- trainControl(method = "cv", number = 10)
tuneGrid <- expand.grid(size = seq(from = 1, to = 10, by = 1), decay = seq(from = 0.1, to = 0.9, by =
0.3))
set.seed(202650) # Adicionei um tuneGrid para trabalhar com os hiperparâmetros.
rna <- train(biomassa~., data=treino, method="nnet", trControl=control, tuneGrid=tuneGrid,
linout=T,
           MaxNWts=10000, maxit=2000, trace=F)
rna

### Predições
predicoes.rna <- predict(rna, teste)

##### CALCULO DE METRICAS #####
# install.packages("Metrics")
library(Metrics)

# Dados reais e preditos
obs <- teste$biomassa
pred <- predicoes.rna
n <- length(obs)
k <- 1 # Graus de liberdade 1 conforme aula do professor

# Métricas de Erro e Correlação
mae_val <- mae(obs, pred)
rmse_val <- rmse(obs, pred)
r_pearson <- cor(obs, pred)

# Erro Padrão da Estimativa (Syx) e Syx%
syx <- sqrt(sum((obs - pred)^2) / (n - k))

# R2 ajustado/simples
r2 <- function(predito, observado) {
  return(1 - (sum((predito-observado)^2) / sum((observado-mean(observado))^2)))
}
r2_val <- r2(predicoes.rna, teste$biomassa)

# Consolidação em um data frame
resultados <- data.frame(
  R2 = r2_val,
  Syx = syx,
  Pearson_r = r_pearson,
  RMSE = rmse_val,
  MAE = mae_val
)

print(resultados)
```

```
##### PREDIÇÃO DE NOVOS CASOS #####
```

```
# Novos dados gerados por IA com valores aleatórios mas que fazem sentido dentro da base  
novos_dados <- read.csv("novos_casos_biomassa.csv", header = TRUE)
```

```
predict.rna <- predict(rna, novos_dados)  
resultado <- cbind(novos_dados, predict.rna)  
View(resultado)
```

```
### GRÁFICO DE RESÍDUOS EM PORCENTAGEM ###
```

```
# Fórmula: ((Volume Real - Volume Estimado) / Volume Real) * 100  
residuos_pct <- ((obs - pred) / obs) * 100
```

```
plot(x = pred, y = residuos_pct,  
     main = "Gráfico de Resíduos (%)",  
     xlab = "Biomassa Estimada",  
     ylab = "Resíduo (%)",  
     pch = 16, col = "darkblue", panel.first = grid())
```

```
abline(h = 0, col = "red", lwd = 2, lty = 2)
```

## AGRUPAMENTO

### Veículo

Lista de Clusters gerados:

| Cluster | Nº de veículos |
|---------|----------------|
| 1       | 75             |
| 2       | 8              |
| 3       | 165            |
| 4       | 126            |
| 5       | 76             |
| 6       | 98             |
| 7       | 93             |
| 8       | 81             |
| 9       | 29             |
| 10      | 95             |
| Total   | 846            |

10 primeiras linhas do arquivo com o cluster correspondente:

| a  | tipo | Cluster |
|----|------|---------|
| 1  | van  | 4       |
| 2  | van  | 4       |
| 3  | saab | 3       |
| 4  | van  | 5       |
| 5  | bus  | 2       |
| 6  | bus  | 9       |
| 7  | bus  | 1       |
| 8  | van  | 5       |
| 9  | van  | 5       |
| 10 | saab | 10      |

Usa 10 clusters no experimento:

Número de clusters utilizados: 10

Colocar a lista de comandos emitidos no RStudio para conseguir os resultados obtidos:

```
# Definir a seed  
set.seed(202650)
```



```
# Ler a base de dados
veiculos <- read.csv("6 - Veiculos - Dados.csv", header = TRUE, sep = ",")

# Remover identificador e variável categórica
dados_cluster <- veiculos[, !(names(veiculos) %in% c("a", "tipo"))]

# Padronizar os dados
dados_padronizados <- scale(dados_cluster)

# Aplicar K-Means com 10 clusters
kmeans_veiculos <- kmeans(
  dados_padronizados,
  centers = 10,
  nstart = 25
)

# Adicionar o cluster à base original
veiculos$cluster <- kmeans_veiculos$cluster

# Visualizar as 10 primeiras linhas
head(veiculos, 10)

# Verificar quantidade de registros na base
nrow(veiculos)

# Verificar quantidade de veículos em cada cluster
table(veiculos$cluster)

# Visualizar as 10 primeiras linhas com seus clusters
veiculos[1:10, c("a", "tipo", "cluster")]

# Listar os veículos pertencentes a cada cluster
split(veiculos$a, veiculos$cluster)
```

## REGRAS DE ASSOCIAÇÃO

### Musculação

### Configuração utilizada

Para a geração das regras de associação da base Musculação, foi utilizado o algoritmo Apriori, por meio do pacote `arules` no RStudio.

Seed: 202650

Algoritmo: Apriori

Suporte mínimo: 0,10 (10%)

Confiança mínima: 0,60 (60%)

Tamanho mínimo da regra: 2 itens

Com essa configuração, foram geradas 102 regras de associação.

### Principais regras geradas

**1. {Adutor} → {Agachamento}**

Suporte: 11,54% | Confiança: 100% | Lift: 3,25

**2. {Adutor, LegPress} → {Agachamento}**

Suporte: 11,54% | Confiança: 100% | Lift: 3,25

**3. {AgachamentoSmith, Bicicleta} → {Extensor}**

Suporte: 30,77% | Confiança: 100% | Lift: 2,00

**4. {Bicicleta, Esteira} → {Extensor}**

Suporte: 38,46% | Confiança: 100% | Lift: 2,00

**5. {AgachamentoSmith, Bicicleta, Esteira} → {Extensor}**

Suporte: 23,08% | Confiança: 100% | Lift: 2,00

**6. {AgachamentoSmith, Bicicleta, Gemeos} → {Extensor}**

Suporte: 15,38% | Confiança: 100% | Lift: 2,00

**7. {AgachamentoSmith, Bicicleta, LegPress} → {Extensor}**

Suporte: 19,23% | Confiança: 100% | Lift: 2,00

**8. {Bicicleta, Esteira, Gemeos} → {Extensor}**

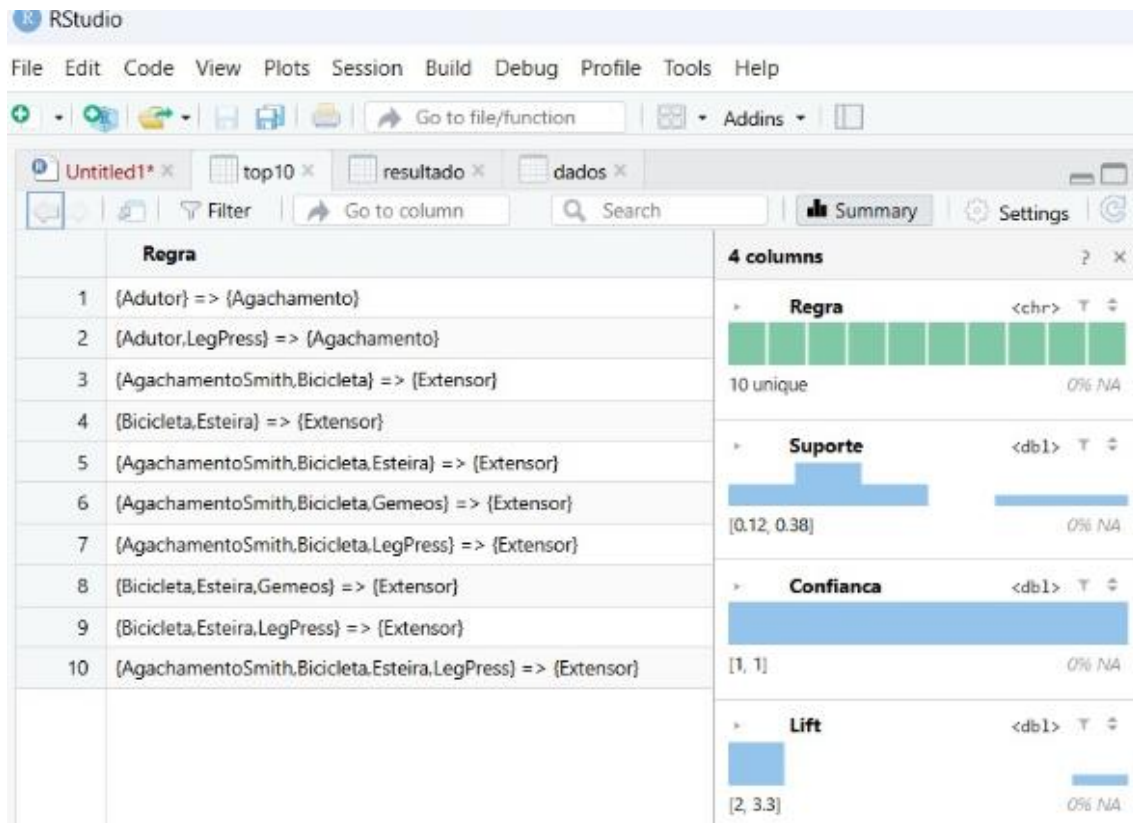
Suporte: 15,38% | Confiança: 100% | Lift: 2,00

**9. {Bicicleta, Esteira, LegPress} → {Extensor}**

Suporte: 23,08% | Confiança: 100% | Lift: 2,00

## 10. {AgachamentoSmith, Bicicleta, Esteira, LegPress} → {Extensor}

Suporte: 15,38% | Confiância: 100% | Lift: 2,00



### Lista de comandos utilizados no RStudio

```
library(arules)
library(readxl)

set.seed(202650)

dados <- read_excel(file.choose(), col_names = FALSE)

dim(dados)
head(dados, 10)
View(dados)

transacoes_lista <- apply(dados, 1, function(x) {
  x <- as.character(x)
  x <- x[!is.na(x)]
  x <- trimws(x)
  x <- x[x != ""]
  unique(x)
})
```

```
})

transacoes_lista <- transacoes_lista[lengths(transacoes_lista) > 0]

transacoes <- as(transacoes_lista, "transactions")

summary(transacoes)
inspect(head(transacoes, 10))

regras <- apriori(
  transacoes,
  parameter = list(
    supp = 0.10,
    conf = 0.60,
    minlen = 2
  )
)

length(regras)
inspect(regras)

regras_conf <- sort(
  regras,
  by = "confidence",
  decreasing = TRUE
)

inspect(head(regras_conf, 10))

regras_lift <- sort(
  regras,
  by = "lift",
  decreasing = TRUE
)

inspect(head(regras_lift, 10))

resultado <- data.frame(
  Regra = labels(regras_lift),
  Suporte = quality(regras_lift)$support,
  Confianca = quality(regras_lift)$confidence,
  Lift = quality(regras_lift)$lift
)

head(resultado, 10)
View(resultado)
```



**UFPR – Universidade Federal do Paraná**  
**Setor de Educação Profissional e Tecnológica**  
**Especialização em Inteligência Artificial Aplicada**  
**Disciplina: Aprendizado de Máquina – Prof Jaime Wojciechowski**



```
write.csv(  
  resultado,  
  "regras_musculacao.csv",  
  row.names = FALSE  
)  
  
top10 <- head(resultado, 10)  
print(top10, row.names = FALSE)  
View(top10)
```